INFOF413 - Heuristic Optimization : Implementation 1 report

Chica Cobo Sofia 518664

17th April 2026

Abstract

This report studies local search methods for the Linear Ordering Problem (LOP). Twelve iterative improvement configurations were compared by combining two initial solutions, two pivoting rules, and three neighborhoods. In addition, two VND variants were evaluated using two different neighborhood orders. To reduce the running time of the local search, delta evaluation was used in order to avoid recomputing the full objective function after each move.

The results show clear differences between the neighborhoods. The insert neighborhood obtained the best solution quality in all configurations, while transpose was systematically the weakest one, although it was by far the fastest. For VND, the order `transpose` → `insert` → `exchange` obtained a slightly better average deviation than `transpose` → `exchange` → `insert`, but the difference was not statistically significant. Overall, the results show that the neighborhood choice has the strongest impact on performance.

1 Introduction

The Linear Ordering Problem (LOP) is a combinatorial optimization problem in which the objective is to find an ordering of elements that maximizes the sum of the coefficients that are compatible with this ordering. This problem is known to be NP-hard and appears in different application domains such as ranking on tournaments, economics, and archeology.

In this work, we study local search methods for the LOP. More precisely, we compare several iterative improvement algorithms obtained by combining different pivoting rules, neighborhoods, and initial solution methods. We also study two variants of Variable Neighbourhood Descent (VND), in order to evaluate the effect of the neighborhood order on the final solution quality.

2 Problem statement

The Linear Ordering Problem can be defined on a square matrix $C = (c_{ij})$ of size $n \times n$. The objective is to find a permutation π of the indices $\{1, \ldots, n\}$ such that the sum of the entries located above the diagonal after permutation is maximized. Formally, the objective function is:

$$f(\pi) = \sum_{i=1}^{n} \sum_{j=i+1}^{n} c_{\pi(i)\pi(j)}.$$

Since the LOP is NP-hard, exact solution methods become difficult to apply on larger instances. For this reason, heuristic methods are often used to obtain good solutions in acceptable computation time. Here, we study local search methods.

3 Methods

First, we implemented 12 iterative improvement algorithms configurations obtained by combining:

1. The initial solution: a random permutation or Chenery-Watanabe heuristic, a constructive heuristic that iteratively selects the most attractive unassigned row

2.The pivoting rule: first-improvement that stops as soon as an improving move is found, or best-improvement that explores the whole neighborhood and selects the best improving move.

3. The neighborhoods:

- *Transpose*: swap two adjacent elements,
- *Exchange*: swap two elements at arbitrary positions,
- *Insert*: remove one element from position i and insert it at position j.

The iterative improvement algorithms follow scheme below:

Algorithm 1 Iterative Improvement

```
π ← GenerateInitialSolution()
while π is not a local optimum do
    choose a neighbour π' ∈ N(π) such that F(π') > F(π)
    π ← π'
end while
```

Then, two VND variants using first-improvement and CW initialization was implemented, with the following neighborhood orders:

- transpose → exchange → insert,
- transpose → insert → exchange.

VND successively applies different neighborhoods to the current solution and, whenever an improving move is found in one neighborhood, restarts the search from the first neighborhood until no neighborhood can further improve the solution.

Delta evaluation

To optimize the implementation, we used *delta evaluation*. The main idea is that after a move, most terms of the objective function remain unchanged. Therefore, instead of recomputing the full objective value from scratch, we only update the terms that are affected by the move.

Let $\Delta = f(\pi') - f(\pi)$ denote the variation in objective value after applying a move.

Transpose. If two adjacent elements a and b are swapped, only their mutual contribution changes. Hence:

$$\Delta_{\text{transpose}} = c_{ba} - c_{ab}.$$

Exchange. If the elements at positions i and j $(i < j)$ are exchanged, the changed contributions are the pair formed by these two elements and their interactions with the elements located between them. If $a = \pi(i)$ and $b = \pi(j)$, then:

$$\Delta_{\text{exchange}} = (c_{ba} - c_{ab}) + \sum_{k=i+1}^{j-1} \left(c_{b\pi(k)} + c_{\pi(k)a} - c_{a\pi(k)} - c_{\pi(k)b}\right).$$

Insert. If the element $a = \pi(i)$ is removed from position i and inserted at position j, only the relative order between a and the crossed elements changes.

If $i < j$, then:

$$\Delta_{\text{insert}} = \sum_{k=i+1}^{j} \left(c_{\pi(k)a} - c_{a\pi(k)}\right).$$

If $i > j$, then:

$$\Delta_{\text{insert}} = \sum_{k=j}^{i-1} \left(c_{a\pi(k)} - c_{\pi(k)a}\right).$$

4 Results

The results presented in this section are based on the 78 benchmark instances of size 150 and 250 considered in the experiments. For each instance, the objective value obtained by an algorithm was compared with the corresponding best known value. This made it possible to compute the relative percentage deviation, which was used as the main indicator of solution quality. The average deviations reported in the following tables were obtained by aggregating these values over all benchmark instances.

4.1 Iterative Improvement

Table 4.1 presents the average results of the 12 iterative improvement configurations over the 78 benchmark instances.

Pivoting	Neighborhood	Init.	Avg. dev. (%)	Std. dev.	Total time (s)	Avg. time/inst. (s)
first	insert	cw	2.0164	0.4109	125.9544	1.6148
first	insert	random	2.0680	0.4398	189.3933	2.4281
best	insert	cw	2.2608	0.4065	34.5371	0.4428
best	insert	random	2.2981	0.4403	38.9556	0.4994
first	exchange	random	2.8590	0.4934	217.2688	2.7855
first	exchange	cw	2.9660	0.4613	173.1395	2.2197
best	exchange	random	3.6359	0.6030	33.6340	0.4312
best	exchange	cw	3.8465	0.6054	31.2967	0.4012
best	transpose	cw	17.0783	1.6065	0.2191	0.0028
first	transpose	cw	17.0863	1.6042	0.2684	0.0034
best	transpose	random	34.0966	3.3375	0.0031	0.0000
first	transpose	random	34.4563	3.8876	0.6266	0.0080

Table 1: Main results for iterative improvement.

The first important observation is that the insert neighborhood dominated the other two in terms of solution quality. In contrast, the fastest neighborhood transpose was by far the weakest neighborhood, especially with random initialization.

Surprisingly, in the present implementation, the best-improvement variants of insert and exchange were faster than the corresponding first-improvement variants, although first-improvement is often expected to reduce the search effort. This suggests that the actual running time is strongly influenced by the implementation details and by the number of iterations needed before reaching a local optimum.

For the initial solution method CW gave better results than random initialization, with a lower mean average deviation; 7.5424% against 13.2357%.

Init.	Neighborhood	first	best	Better one
cw	exchange	2.9660	3.8465	first
cw	insert	2.0164	2.2608	first
cw	transpose	17.0863	17.0783	best
random	exchange	2.8590	3.6359	first
random	insert	2.0680	2.2981	first
random	transpose	34.4563	34.0966	best

Table 2: Effect of the pivoting rule on average deviation (%).

The comparison by pivoting rule is also interesting. For insert and exchange, first-improvement obtained better average deviations than best-improvement, whereas for transpose the two pivoting rules produced almost identical results. This can be seen in Table 4.1.

Pivoting	Init.	transpose	exchange	insert
best	cw	17.0783	3.8465	2.2608
best	random	34.0966	3.6359	2.2981
first	cw	17.0863	2.9660	2.0164
first	random	34.4563	2.8590	2.0680

Table 3: Effect of the neighborhood on average deviation (%).

The comparison by neighborhood in 4.1 confirms the efficiency of insert in all four combinations of pivoting rule and initialization. The difference between `first + insert + cw` and the transpose- or exchange-based algorithms was statistically significant. However, the difference between `first + insert + cw` and `first + insert + random` was not significant at the 5% level, which suggests that these two insert-based variants behave very similarly.

Compared algorithms	Significant t-test	Significant Wilcoxon
`first_insert_random` vs `first_transpose_cw`	True	True
`best_transpose_cw` vs `first_exchange_random`	True	True
...	...	...
`first_insert_cw` vs `best_insert_random`	False	False
`first_exchange_random` vs `first_exchange_cw`	False	False
`best_transpose_random` vs `first_transpose_random`	False	False
`best_insert_cw` vs `best_insert_random`	False	False

Table 4: Examples of pairwise statistical comparisons between iterative improvement algorithms.($\alpha = 0.05$)

The pairwise statistical comparisons were made between two algorithms at a time among the 12 iterative improvement configurations. These comparisons show that most of the algorithms are significantly different in terms of solution quality. Out of the 66 possible pairwise comparisons, 62 were significant with the paired t-test and 61 with the Wilcoxon signed-rank test. The largest differences appear between the insert-based methods and the transpose-based methods.

4.2 VND

We compared two VND variants:

- *order1*: transpose → exchange → insert,
- *order2*: transpose → insert → exchange.

The descriptive statistics that were made over the 78 benchmark instances are shown in Table 4.2.

Statistic	Value
Mean delta order1 (%)	2.063339
Mean delta order2 (%)	1.995886
Mean difference (order1 - order2)	0.067453
Median difference (order1 - order2)	-0.005187

Table 5: Descriptive statistics for the two VND variants.

Order2 obtained a slightly better average deviation than order1. However, the difference was very small. To determine whether this difference was statistically significant, we applied a paired t-test and a Wilcoxon signed-rank test.

Test	Statistic	p-value
Paired t-test	1.254192	0.213566
Wilcoxon signed-rank test	1377.0	0.415443

Table 6: Statistical tests for the VND comparison ($\alpha = 0.05$).

Both p-values are larger than 0.05. Therefore, we cannot conclude that one VND ordering is significantly better than the other. In practice, the two variants appear to have very similar performance.

4.3 Conclusion

This report compared local search methods for the Linear Ordering Problem. Twelve iterative improvement configurations and two VND variants were evaluated on the benchmark instances.

The results show that the neighborhood choice has the strongest effect on performance. Insert obtained the best solution quality in all configurations, while transpose was the fastest but also the weakest. Exchange gave intermediate results. The CW initialization generally improved the results compared to random initialization.

The statistical tests confirm these observations. Most pairwise comparisons between the iterative improvement configurations were significant, especially when comparing insert-based methods with transpose-based methods. For VND, the order `transpose` → `insert` → `exchange` obtained a slightly better average deviation, but the difference was not statistically significant.

Overall, the experiments show that insert is the most effective neighborhood among the three tested ones, and that an efficient implementation with delta evaluation is important to keep the computation time reasonable.